

تقرير برمجة تفرعية

نظام إدارة المتجر الإلكتروني

E-Commerce Management System — Django REST Framework



الطلاب :

أحمد مؤيد علوش

نوران محمد كندر

ريما أسامة آغي

رضا احمد عارف

وئام وسيم حسون

بإشراف

م.حذيفة عقيل

أولاً: شرح تضارب البيانات :

عملية الشراء وتحديث المخزون ((Create Order with Stock))

أ) المشكلة الأساسية ((The Race Condition))

في أنظمة التجارة الإلكترونية عالية الحمل، تتم عملية تحديث المخزون برمجياً عبر قاعدة البيانات عبر ثلاث خطوات متتالية:

قراءة القيمة الحالية من الداتايز

تعديل القيمة في ذاكرة الخادم

حفظ القيمة الجديدة في الداتايز

المشكلة ((Thread Overlapping)) عندما يضغط مئات المستخدمين على زر "شراء" في نفس الأجزاء من الثانية لمنتج متبقي منه قطعة واحدة فقط، تقرأ الخيوط (Threads) المتزامنة الخاصة بالمستخدمين نفس القيمة الحالية في نفس الوقت ($Stock = 1$).

تضارب البيانات ((Lost Update / Dirty Read)) قبل أن يقوم الخيط الأول بحفظ تعديله، يقرأ الخيط الثاني نفس القيمة القديمة. يقوم الخيط الأول بالخصم والحفظ ($Stock = 0$))، ويقوم الخيط الثاني بالخصم والحفظ أيضاً بناءً على القراءة القديمة.

النتيجة الكارثية ((Overselling)) ينجح كلا المستخدمين في إتمام الطلب والدفع، ويتحول المخزون إلى قيمة سالبة أو غير منطقية، وهو ما يُعرف بـ البيع الزائد ((Overselling))، مما يتسبب في انهيار مصداقية النظام وتضارب البيانات المالية والمخزنية.

حل المشكلة :

● القفل الموزع على مستوى الذاكرة ((Distributed Cache Lock))

```
lock_key = f"checkout_lock_user_{customer_name}"
with cache.lock(lock_key, timeout=30, blocking_timeout=10)
```

: هذا القفل يعمل على مستوى نظام Redis/Cache (أعلى من قاعدة البيانات). هو قفل مخصص لكل مستخدم على حدة

يمنع نفس المستخدم من إرسال طلبات "Checkout" متكررة ومزدوجة في نفس الثواني (بسبب النقر المتكرر على زر الشراء مثلاً)، حيث يجبر الطلب الثاني على الانتظار حتى ينتهي الطلب الأول تماماً، مما يحمي النظام من استنزاف الموارد والطلبات العشوائية للمستخدم نفسه.

● استراتيجيات قاعدة البيانات ((Database Concurrency Strategies))

لحماية المخزون الفعلي للمنتجات من خيوط المستخدمين المختلفين، تم بناء الكود ليدعم 3 استراتيجيات هندسية متطورة يتم التبديل بينها ديناميكياً

استراتيجية التحديث الذري المباشر (Atomic Database Updates)

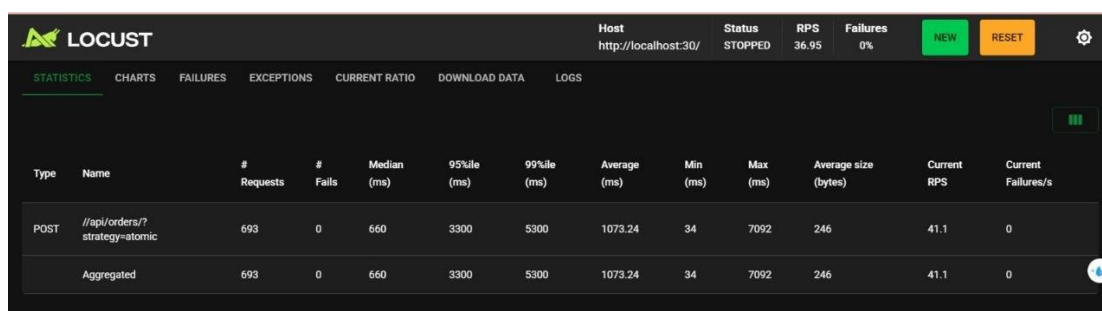
تم تطبيقها في التابع update_stock_Atomic باستخدام تعبيرات الـ F expressions

بدلاً من سحب قيمة المخزون إلى الذاكرة في الخادم، تعديلها، ثم حفظها، يتم إرسال العملية الحسابية مباشرة ليتم تنفيذها داخل محرك قاعدة البيانات كآمر واحد غير قابل للتجزئة:

$$\{UPDATE products SET stock = stock - quantity WHERE id = product_id AND stock \geq \{ quantity\}$$

قاعدة البيانات تنفذ هذا الأمر بشكل ذري (Atomic)، فإذا كان المخزون أقل من الكمية المطلوبة، لن تتأثر أي صفوف (rows == 0) ويرمي النظام استثناءً فوراً.

أدائها خارق وسريع جداً لأنها لا تضع أقفالاً طويلة، لكن عيبها أنها لا تعطيك مرونة برمجية للتحقق من شروط معقدة في الذاكرة (In-Memory Validation) قبل التحديث.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/orders/?strategy=atomic	693	0	660	3300	5300	1073.24	34	7092	246	41.1	0
	Aggregated	693	0	660	3300	5300	1073.24	34	7092	246	41.1	0

استراتيجية القفل المتفائل (Optimistic Locking)

تم تطبيقها في التابع update_stock_optimistic بالاعتماد على حقل رقم الإصدار: version:

تفترض هذه الاستراتيجية أن التعارض نادر الحدوث. يقرأ الخيط بيانات المنتج ورقم إصداره الحالي مثلاً version=5. وعند الحفظ، يشترط في عبارة الـ WHERE أن يكون الإصدار ما زال: 5:

$$\{WHERE id = product_id AND version = 5\}$$

إذا نجح التحديث، يتم رفع الإصدار إلى 6.

إذا قام خيط آخر بتعديل المنتج خلال جزء من الثانية، سيتغير رقم الإصدار في الداتايز ليصبح 6. بالتالي، عندما يحاول الخيط الأول الحفظ، لن يجد صفّاً يطابق الشرط (updated_count == 0). هنا يتدخل الكود لتطبيق خوارزمية التراجع العشوائي الذكي (Smart Random Backoff) عبر `time.sleep(random.uniform(0.01, 0.05))` ويعيد المحاولة حتى 3 مرات.

ممتازة للأنظمة التي يقل فيها التضارب، ولكن في حالات الضغط الشديد جداً مثل عروض Black Friday على منتج واحد ستفشل المحاولات الثلاث بسبب كثرة التضاربات المتتالية، مما يضطر النظام لرمي استثناء للمستخدم (تنخفض الـ UX).



Host

http://localhost:30/

Status

SPAWNING

Users

90

RPS

29.51

Failures

0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	//api/orders/?strategy=optimistic	533	0	590	4200	6000	1085.28	36	9871	246	33.4	0
	Aggregated	533	0	590	4200	6000	1085.28	36	9871	246	33.4	0



استراتيجية القفل التشاؤمي (Pessimistic Locking)

تم تطبيقها في التابع update_stock_pessimistic باستخدام الدالة الفعالة select_for_update():

تفترض هذه الاستراتيجية أن التضارب حتمي وسيحصل بالتأكيد. بمجرد أن يبدأ الخيط بقراءة المنتج، يتم إطلاق أمر صريح لقاعدة البيانات:

{SELECT ... FOR UPDATE}

تقوم قاعدة البيانات فوراً بحجز وقفل السطر الخاص بهذا المنتج بقفل صارم على مستوى السطرأي خيط آخر يحاول مجرد القراءة أو التعديل لنفس المنتج سيتم إيقافه وإجباره على الانتظار في طابور منظم (Queue) حتى تنتهي المعاملة الحالية تماماً (transaction.atomic) ويتم عمل Commit أو Rollback.

تضمن حماية مطلقة وأماناً بنسبة 100% للبيانات المالية والمخزنية الحرجة من أول محاولة. عيبها الوحيد أنها قد تقلل من سرعة الاستجابة الإجمالية للنظام (Throughput) إذا طالت فترة القفل بسبب طوابير الانتظار.

 LOCUST

Host

http://localhost:30/

Status

STOPPED

RPS

34.86

Failures

0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	//api/orders/?strategy=pessimistic	485	0	730	2600	3200	859	34	3293	246	38.6	0
	Aggregated	485	0	730	2600	3200	859	34	3293	246	38.6	0



تعديل محتويات طلب معلق (Update Order Items)

إذا كان الطلب معلقاً (pending) وقام المستخدم بتعديل محتوياته (إضافة أو حذف منتجات، أو تعديل الكميات) من متصفحين أو من خلال نقرات متزامنة، أو في نفس الوقت الذي يحاول فيه النظام الآلي (أو الآدمن) مراجعة الطلب أو إلغائه. دون قفل، يتم حساب إجمالي السعر بشكل خاطئ، أو يتم إرجاع مخزون قاعدة البيانات مرتين، أو خصم من المحفظة بشكل مزدوج، مما يتسبب في عدم تطابق القيمة المالية المحسوبة مع إجمالي المنتجات الفعلي.

قفل على مستوى الكاش للطلب:

يتم إنشاء قفل فريد لكل طلب (order_update_lock_{order_id}) يمنع تعديل الطلب من أي خيط آخر لمدة 20 ثانية كحد أقصى.

قفل تشاؤمي على الطلب والمستخدم:

يتم استخدام `select_for_update()` لجلب الطلب وجلب بيانات المستخدم (User). هذا يضمن تجميد السجلات بالكامل حتى يتم حساب الفروقات في الكميات، تعديل المخزون عبر الدالة الذرية، وتحديث محفظة المستخدم بالخصم أو الإرجاع (`_deduct_wallet`).

ثانياً: إدارة الموارد الحاسوبية والتحكم بالطاقة الاستيعابية

(Resource Management & Capacity Control)

اختناق الخادم واستنزاف الذاكرة ((Memory & DB Exhaustion)

في الأنظمة العشوائية التي لا تفرض قيوداً صارمة على مواردها، يؤدي تعرض النظام لضغط مفاجئ وهجمات متزامنة من مئات المستخدمين إلى ثلاثة اختناقات كارثية:

استنزاف اتصالات قاعدة البيانات ((Connection Pool Exhaustion يقوم الخادم بفتح وإغلاق اتصال صريح لكل عملية طلب برمجية (Request). وبما أن إنشاء الاتصال عملية مكلفة جداً حاسوبياً، تنهار قاعدة البيانات تحت الضغط وتطلق استثناء الذاكرة الشهير (`Too many connections`).

انفجار الذاكرة العشوائية ((Memory Bloat عند طلب تصفح المنتجات دون قيود، يقرأ الخادم آلاف الأسطر دفعة واحدة ويحولها إلى كائنات (Objects) ويحشرها في الذاكرة العشوائية (RAM)، مما يتسبب بخروج السيرفر عن الخدمة فوراً نتيجة نفاد الذاكرة (`Out of Memory`).

انهيار المعالج بسبب المهام المتراكمة: معالجة المهام الثقيلة (مثل إصدار التقارير أو إرسال إيميلات التأكيد الفورية) في مسار الطلب الرئيسي (Main Request Thread) تستنزف خيوط المعالج وتجعل زمن استجابة النظام للمستخدمين الآخرين شبه منعدم.

التقنيات الهندسية المعتمدة لحل المشكلة :

استدامة اتصالات قاعدة البيانات وتقييد عمرها (`CONN_MAX_AGE = 60`)

بدلاً من تدمير الاتصال بعد كل طلب وإعادة بنائه، تم ضبط الخيار للحفاظ على استمرارية الاتصال وبقائه مفتوحاً لإعادة استخدامه للطلب التالي خلال هذه الدقيقة. هذا يمنع عبء الـ TCP Handshake المتكرر، ويخفف الحمل الحسابي عن المعالج والذاكرة في الـ PostgreSQL.

`'CONN_MAX_AGE': 60,`

التحكم في ذاكرة الـ RAM وحجم الشبكة عبر الـ Pagination

حصر سحب البيانات من قاعدة البيانات بـ 10 منتجات فقط لكل طلب. هذا يحمي ذاكرة الخادم العشوائية (RAM) من التضخم (Memory Bloating)، ويجعل حجم حزم البيانات المرسل عبر الشبكة (Payload) خفيفاً وثابتاً، مما يحسن زمن الاستجابة (Latency).

التقييد ومحددات المعدل (Rate Limiting / Throttling)

لحماية النظام وقاعدة البيانات من خطر الاستنزاف المتعمد الناتج عن هجمات الإغراق (Brute Force / DDOS)، أو السلوك العشوائي المتكرر من قبل مستخدم واحد، تم تضمين سياسة تقييد برمجية تضع سقفاً للعمليات المسموح بها لكل عنوان IP خلال وحدة زمنية معينة. هذا يضمن التوزيع العادل للموارد (Fairness) ويمنع حالة حرمان الموارد الحاد (Resource Starvation) للمستخدمين الآخرين.

حماية الذاكرة المؤقتة من السقوط (2 = SOCKET_TIMEOUT)

وضع قيود زمنية صارمة للاتصال بـ Redis. إذا واجه سيرفر الكاش ضغطاً كبيراً أو تأخراً، فلن تنتظر خيوط النظام طويلاً وحجز الذاكرة بلا فائدة، بل يتم قطع الاتصال بعد ثانيتين (Fallback) ومتابعة القراءة من قاعدة البيانات لضمان استمرارية الخدمة (High Availability).

التحكم بالطاقة الاستيعابية للـ Workers وعزل المهام الخلفية

تم عزل المهام الثقيلة مثل إرسال إيميلات تأكيد الشراء باستخدام Celery بعيداً عن مسار الطلب الرئيسي للمستخدم لتتم معالجتها في الخلفية (Asynchronous). تم تقييد عدد العمليات المتوازية للـ Worker بـ 4 مهام كحد أقصى موازٍ لأنوية المعالج المتاحة لمنع استهلاك السيرفر، مع تفعيل خاصية الـ Prefetch بمقدار 1 لمنع الـ Workers من سحب مهام كثيرة واحتكارها في الذاكرة دون تنفيذ، مما يضمن توزيعاً عادلاً للمهام Fair Distribution

إدارة نموذج الخيوط في بيئة الإنتاج الفعلي (Thread Pool Management via Waitress)

في بيئة التشغيل الفعلي، لا يعمل Django بمفرده لاستقبال الطلبات، بل يتم تخديمه بواسطة خادم ويب إنتاجي Waitress عبر:

```
waitress-serve --port=8000 --backlog=60 my_site.wsgi:application
```

ومن خلاله يتم التحكم بالطاقة الاستيعابية وحماية الموارد عبر تقنيتين:

حوض الخيوط المخصص ((Worker Thread Pool)) يقوم خادم Waitress عند الإقلاع بإنشاء قنوات معالجة مستقلة متوازية (Multi-threaded Architecture). وبدلاً من استهلاك الذاكرة عبر فتح عمليات (Processes) منفصلة بالكامل، يتم الاعتماد على الخيوط التي تتشارك نفس مساحة الذاكرة للتطبيق. هذا يعتبر الحل الأمثل للعمليات التي تتطلب انتظاراً طويلاً لمدخلات ومخرجات البيانات (I/O Bound Tasks)، مثل فترات استعلام واسترجاع البيانات من قاعدة بيانات PostgreSQL، حيث يمكن للخيوط الأخرى متابعة تخديم الزبائن دون تجميد النظام، مما يضع سقفاً ثابتاً (Fixed Thread Pool) يضمن تقييد استهلاك الـ CPU والـ RAM بحدود آمنة ومعروفة مسبقاً.

نظام طوابير الانتظار ((Request Queuing / Buffer Area)) عندما يتجاوز عدد المستخدمين المتزامنين سقف خيوط المعالجة المتاحة، لا ينهار النظام ولا تُرفض الطلبات، بل تعمل الـ backlog المحددة بالقيمة 60 كمنطقة تخزين مؤقت (Buffer Area). تحتفظ هذه المنطقة بالطلبات الزائدة لفترة وجيزة جداً حتى يتحرر أول خيط معالجة داخل حوض التطبيق ليقوم بأخذ الطلب الذي يليه. هذا الإجراء يمنع انهيار السيرفر ويحد تماماً من ظاهرة عبء تبديل السياق المفرط (Context Switching Overhead) التي تحدث عندما يحاول المعالج التبديل العشوائي والمتزامن بين آلاف المهام في نفس الوقت.

ثالثاً: المعالجة غير المتزامنة لتقليل زمن الاستجابة (Asynchronous Queues & Low Latency)

المشكلة الأساسية: تجميد خيوط المعالجة وال High Latency

في بنية الويب التقليدية، إذا جعل الخادم ينفذ مهاماً جانبية ثقيلة مثل الاتصال بخوادم البريد الإلكتروني الخارجية لإرسال تأكيد الطلب، أو توليد فواتير PDF ضمن مسار طلب ال HTTP الرئيسي للمستخدم:

- تأخير زمن الاستجابة: سينتظر المستخدم على شاشة التحميل (Loading) لعدة ثوانٍ حتى تنتهي شبكة خادم البريد من الاستجابة.
- احتكار خيوط السيرفر: خلال هذه الثواني، يتم تجميد خيوط المعالجة وحرمانه من تخدم زبائن آخرين، مما يرفع من فرصة انهيار السيرفر تحت ضغط الطلبات المتزامنة.

الحل البرمجي: فصل المهام عبر (Celery & Redis)

تعتمد البنية الهيكلية للنظام على فصل العمليات المرتبطة بطلب ال HTTP الفوري عن العمليات الثقيلة، وذلك للحفاظ على زمن استجابة ضئيل جداً للمستخدمين (Low Latency). بمجرد إتمام الشراء، يرد النظام فوراً على الزبائن بـ "تم استلام طلبك بنجاح"، ويتم قذف المهمة الثقيلة إلى طابور (Redis Message Broker) ليتولى عمال (Celery Workers) معالجتها في الخلفية دون إشغال مسار التطبيق الرئيسي.

```
@shared_task(bind=True, max_retries=3, default_retry_delay=60, rate_limit='10/m')
```

```
def send_order_confirmation_email(self, order_id, customer_email, customer_name, total_price):
```

رابعاً معالجة البيانات الضخمة على دفعات (Batch Processing & Memory Recycling)

المشكلة الأساسية: انفجار الذاكرة وشل الأداء (Memory Bloat & Table Locking)

Locking)

عند طلب تقرير جرد للمخزون أو حساب مبيعات دوري يضم عشرات آلاف السجلات بالطريقة التقليدية (غير سحب الكائنات دفعة واحدة)، يواجه النظام الكوارث التالية:

تضخم الذاكرة وحالة ال OOM: يقوم محرك (Django ORM) بسحب كافة السجلات وتحويلها إلى كائنات في الذاكرة العشوائية (RAM) دفعة واحدة. هذا يسبب قفزة حادة (Memory Spike) تؤدي إلى انهيار الخادم وخروجه عن الخدمة فوراً بسبب تجاوز الحد المسموح به. (Out of Memory)

قفل الجداول (Exclusive Table Lock): الاستعلامات الضخمة تضع ضغطاً هائلاً على قاعدة البيانات (PostgreSQL)، مما يضطرها لتطبيق قفل على الجداول لحماية سلامة البيانات أثناء القراءة الطويلة. نتيجة

لذلك، أي مستخدم يحاول إتمام عملية شراء في تلك اللحظة سيعلق طلبه (Hanging) ويواجه الموقع حالة جمود كاملة.

تداخل المهام المتزامنة (Task Overlapping): إذا استغرقت مهمة بناء التقرير وقتاً طويلاً وقام النظام بإعادة إطلاقها تلقائياً قبل تفريغ الأولى، ستتداخل المهام وتستهلك موارد السيرفر بشكل مزدوج حتى يسقط تماماً.

التقنيات الهندسية المعتمدة في الكود لحل المشكلة:

آلية التدفق المجزأ (Queryset Iteration via Chunks): بدلاً من تحميل عشرات الآلاف من السجلات دفعة واحدة في الـ RAM، تم ضبط الكود ليعتمد على خاصية `iterator(chunk_size=500)`. هذه التقنية تجبر النظام على سحب البيانات من قاعدة البيانات على هيئة دفعات صغيرة ومتتالية (بمعدل 500 سجل في كل دفعة). بمجرد انتهاء معالجة الدفعة وتحويلها لأسطر نصية، يتم تحرير كائنات الـ Django فوراً من الذاكرة لاستقبال الدفعة التالية، مما يضمن ثبات خط استهلاك الذاكرة العشوائية مهما تضخم حجم جدول المنتجات.

التدفق النصي في الذاكرة المؤقتة (In-Memory Data Streaming): لحماية وحدات التخزين الفيزيائية (Disk Storage) من الإجهاد الناتج عن عمليات الكتابة والقراءة المتكررة للملفات، يتبنى النظام تقنية الـ `io.StringIO` و `io.BytesIO` ليتم توليد أسطر الـ CSV ومخططات البيانات (Matplotlib Charts) وضخها مباشرة داخل هذه الكتل كـ `Streams` (تدفقات بيانات خفيفة) دون الحاجة لإنشاء ملفات حقيقية على السيرفر.

قفل منع التكرار والضمان الفردي للمهام (Idempotency Lock via Redis): لتجنب كارثة تداخل خيوط المعالجة لنفس التقرير، تم بناء آلية حماية ذكية باستخدام الـ `Cache` المدعوم بـ `Redis`. عبر تعليمة `cache.delete` في بلوك الـ `finally` الشامل، يضمن النظام ذرياً (Atomic) ألا يدار التقرير بواسطة خيطين بالتزامن. إذا حاول السيرفر تشغيل نفس المهمة أثناء عملها، يتم إسقاط الطلب الجديد فوراً (Skipping) لحماية موارد النظام، مع ضمان تحرير القفل دائماً حتى لو فشلت المهمة أو واجهت خطأ غير متوقع.

خامساً : سلامة المعاملات (Transaction Integrity / ACID)

المشكلة الأساسية (The Concurrency & Data Integrity Challenge)

في أنظمة التجارة الإلكترونية، لا يمكن النظر إلى عملية الشراء على أنها خطوة واحدة منفصلة، بل هي عبارة عن عملية مركبة (Compound Transaction) تتكون من عدة خطوات جوهرية متتالية:

1. التحقق من رصيد محفظة المستخدم وخصمه (`deduct_wallet_`).
2. إنشاء سجل الطلب الأساسي (`Order.objects.create`).
3. تحديث المخزون لكل منتج داخل الطلب عبر الاستراتيجيات المختلفة (`update_stock`).
4. إنشاء سجلات عناصر الطلب (`OrderItem.objects.create`).
5. تفريغ سلة التسوق الخاصة بالمستخدم (`Cart.objects.delete`).

(Partial Failures & Inconsistency) إذا تم تنفيذ هذه الخطوات بشكل منفصل بدون آلية حماية، وحصل خطأ مفاجئ في منتصف العملية (مثل: انقطاع اتصال قاعدة البيانات، نفاذ مخزون أحد المنتجات في اللحظة الأخيرة، أو عدم كفاية رصيد المحفظة)، ستحدث كارثة في البيانات. فقد يتم خصم المال من محفظة

المستخدم دون إنشاء الطلب، أو قد يتم إنقاص المخزون دون تسجيل الفاتورة. هذا الخلل يضرب مبدأ سلامة البيانات (Data Integrity) في مقتل.

حل المشكلة وتطبيق مبادئ ACID

لضمان ألا تقع قاعدة البيانات في حالة "التشوه" أو عدم الاتساق، تم بناء النظام ليدعم خصائص ACID (التحقق، الاتساق، العزل، والمتانة) برمجياً عبر التقنيات التالية:

خاصية الذرية (Atomicity): "الكل أو لا شيء"

تم استخدام مدير السياق `with transaction.atomic()` لتغليف عملية الشراء بالكامل داخل المعاملة. يضمن هذا السطر أن جميع العمليات المركبة (دفع + تحديث مخزون + إنشاء طلب + حذف السلة) تُعامل ككتلة واحدة غير قابلة للتجزئة.

في حالة النجاح: يتم اعتماد جميع التغييرات معاً في قاعدة البيانات دفعة واحدة (Commit).

في حالة الفشل: إذا رُبي أي استثناء (`ValueError` أو `DatabaseError`) بسبب نفاد مخزون أو فشل منطقي، يقوم النظام فوراً بعملية تراجع شاملة (`Rollback`)، مما يلغي أي خطوة تم تنفيذها مسبقاً (مثل تراجع خصم المحفظة).

العزل والتحكم بالوصول المتزامن (Isolation & Concurrency Control)

تحت ظروف الوصول المتزامن الشديد (`High Concurrent Access`) من قِبل مئات المستخدمين في نفس اللحظة، يفرض الكود عزلاً صارماً عبر دمج طبقتين من الأقفال:

قفل الكاش الموزع (cache.lock): يمنع النقرات المتزامنة للمستخدم نفسه من توليد طلبات شراء مزدوجة ومكررة.

القفل التشاؤمي (select_for_update()): عند اختيار استراتيجية `pessimistic`، يتم قفل أسطر المنتجات وسجل المستخدم في قاعدة البيانات فوراً. هذا يمنع أي خيط (`Thread`) آخر من التعديل على رصيد المستخدم أو مخزون المنتج حتى تنتهي المعاملة الحالية تماماً، مما يقضي على مشكلة القراءة غير النظيفة (`Dirty Read`) والتحديث المفقود (`Lost Update`).

آلية التراجع الذكي وإعادة المحاولة (Smart Retry Mechanism)

عند حدوث تضارب في البيانات نتيجة الضغط العالي (خاصة عند استخدام القفل المتفائل `optimistic`)، لا يقوم النظام برمي خطأ فوراً ليفسد تجربة المستخدم (UX)، بل يمر عبر حلقة تكرار ذكية

يعيد الكود المحاولة حتى 3 مرات، ويفصل بينها بفترة زمنية قصيرة لمنح قاعدة البيانات فرصة لإنهاء المعاملات المتشابكة، مما يوازن بدقة بين صرامة الأمان وسلاسة الأداء.

فصل العمليات الملحقة (Non-Blocking Side Effects)

لضمان سرعة استجابة النظام (`Low Latency`) وعدم تعليق المعاملة المالية بسبب عمليات ثانوية مثل إرسال بريد إلكتروني، تم استخدام دالة `transaction.on_commit()`

يضمن الكود عدم إرسال إيميل تأكيد الطلب إلى المستخدم إلا بعد نجاح المعاملة بالكامل واعتمادها في الداتابيز. فإذا فشلت المعاملة وحصل `Rollback`، لن يتم إرسال الإيميل، مما يمنع تضارب معلومات النظام مع ما يصل للمستخدم.

سادسا : استراتيجية التخزين المؤقت الموزع Distributed Caching

تعتبر قاعدة البيانات (Database) هي الاختناق الأكبر (Bottleneck) في الأنظمة عالية الأداء عند تعرضها لآلاف الطلبات المتزامنة لقراءة نفس البيانات. للتغلب على هذه المشكلة، تم دمج طبقة تخزين مؤقت موزع (Distributed Cache) باستخدام **Redis** في نظام المنتجات والمبيعات لحماية قاعدة البيانات وتسريع زمن الاستجابة (Latency) إلى أجزاء من الملي ثانية.

بنية ذاكرة التخزين وعمر البيانات ((Cache Topology & TTL)

تم تقسيم البيانات داخل Redis بناءً على طبيعة استهلاكها ودرجة تغيرها عبر مفاتيح فريدة (Namespaced Keys) مع تحديد وقت صلاحية ذكي لكل نوع لمنع استهلاك الذاكرة العشوائية (RAM Consumption):

بيانات المنتج الفردي (**product:{id}**): يتم كاش للبيانات الكاملة للمنتج بعد تحويلها لقاموس عبر ال **Serializer** بمهلة صلاحية **PRODUCT_CACHE_TTL = 900** ثانية (15 دقيقة).

المنتجات الأكثر رواجاً (**products:trending**): تُخزن القائمة الجاهزة بمهلة صلاحية **TRENDING_CACHE_TTL = 600** ثانية (10 دقائق) نظراً لأن الحسابات التجميعية للمبيعات ثقيلة جداً.

المنتجات الأكثر مشاهدة (**products:most_viewed**): تُخزن القائمة بمهلة صلاحية **MOST_VIEWED_CACHE_KEY = 300** ثانية (5 دقائق).

الآليات والتقنيات الهندسية المطبقة

القراءة والكتابة الذكية بالتوازي ((Cache-Aside Pattern & Bulk Operations)

تم تطبيق نمط ال Cache-Aside في جلب المنتج الفردي؛ حيث يتدفق الطلب أولاً إلى الذاكرة السريعة، وفي حال عدم وجوده (Cache Miss) يتم قراءته من قاعدة البيانات وكتابته فوراً في الكاش للطلبات اللاحقة.

وأيضاً في تابع العرض الجماعي (**ProductViewSet.list**)؛ فبدلاً من إطلاق استعلامات فردية للكاش مما يتسبب في مشكلة ال Network Round-Trip Overkill، تم استخدام الأوامر الجماعية

```
cached_products_dict = cache.get_many(cache_keys)
```

```
cache.set_many(products_to_cache, timeout=900)
```

العدادات الذرية اللحظية ((Atomic In-Memory Counters)

تتبع المنتجات الأكثر مشاهدة في قواعد البيانات التقليدية يتطلب أمر **UPDATE** مع كل نقرة مستخدم، وهو ما يدمر أداء النظام. تم حل هذه المعضلة هندسياً عبر التابع **track_product_view** بالاعتماد على عدادات Redis الذرية المتزامنة بالكامل والخفيفة جد

يتم زيادة العداد في الذاكرة بـ 10 دون لمس قاعدة البيانات، وعند طلب قائمة الأكثر مشاهدة يتم تجميع هذه العدادات وفرزها في الذاكرة، ثم كاش النتيجة النهائية لـ 5 دقائق لحماية المعالج.

اتساق البيانات وإلغاء الصلاحية الفوري (Cache Invalidation & Consistency)

المشكلة الأكبر في التخزين المؤقت هي قراءة بيانات قديمة (Stale Data). لضمان الاتساق القوي (Strong Consistency) لبيانات المنتجات، تم ربط أي عملية تعديل على البيانات برمجياً بإلغاء الصلاحية الفوري (Eviction):

عند تعديل المنتج أو حذفه أو تحديث مخزونه بأي استراتيجية، يتم إطلاق التابع الصارم `ProductService._invalidate(product_id)` لمسح المفتاح فوراً من Redis، مما يجبر الطلب التالي على قراءة البيانات المحدثة من الداتابيز وكاشها مجدداً.

حماية الذاكرة من العمليات الثقيلة (Anti-Thundering Herd Concept)

في مهام Celery الخلفية `daily_sales_batch_processing` و `generate_weekly_report` التي تقوم بعمل جرد ضخ و بناء ملفات CSV ورسومات بيانية مستهلكة للموارد، تم وضع آلية حماية تمنع تكرار تشغيل المهمة إذا استغرقت وقتاً طويلاً

سابعاً : استراتيجية توزيع الأحمال وضمان الإتاحة العالية (Load Distribution & High Availability)

عندما يتعرض النظام لذروة مفاجئة في أعداد المستخدمين المتزامنين (Traffic Spikes)، فإن الاعتماد على خادم ويب واحد (Single Server) يصبح أكبر نقطة فشل مركزية (Single Point of Failure). لحماية النظام من الانهيار وتحقيق استقرار كامل للخدمة، تم تصميم معمارية توزيع أحمال ذكية تضمن مرونة النظام وقابليته للتوسع الأفقي (Horizontal Scaling).

بنية النظام وتدفق الطلبات (Architecture Topology)

تمت إعادة هيكلة تدفق البيانات برمجياً بحيث لا تتصل المتصفحات أو التطبيقات الخارجية بخوادم Django مباشرة، بل تمر بمحطة فحص وتوجيه مركزية تقف في الخط الأمامي للنظام لإدارة الجلسات والطلبات بكفاءة.

الآليات والتقنيات الهندسية المطبقة في المعمارية

. كسر اختناق الخادم المفرد (Eliminating Server Bottleneck)

في المعماريات التقليدية، تصب جميع طلبات الـ HTTP القادمة من المستخدمين في خادم تطبيق واحد. تحت الضغط العالي، يؤدي ذلك إلى استنزاف الموارد بالكامل؛ حيث تقفز نسبة استهلاك المعالج (CPU) والذاكرة العشوائية (RAM) إلى 100%، مما يتسبب في بطء قاتل في الاستجابة أو خروج السيرفر عن الخدمة تماماً (Server Crash).

آلية موازنة الأحمال والوكيل العكسي (Smart Load Balancing & Reverse Proxying)

لحظر هذا الاختناق نهائياً، تم إدخال خادم **Nginx** ليعمل كوحدة موازنة أحمال أمامية (Upstream Load Balancer) ووكيل عكسي (Reverse Proxy):

يتلقى Nginx كافة الطلبات الخارجية، ويقوم بتصفيتها وحمايتها قبل تمريرها للداخل.

(Round Robin) يقوم Nginx بتوزيع طلبات الـ HTTP برمجياً على خوادم التطبيق الخلفية (Backend Django Workers) بالتناوب ووفق دورة منتظمة. هذا التقنيت يتضمن تقسيم العبء الحسابي بالتساوي بحيث لا يواجه أي خادم مفرد ضغطاً يدفعه للانهييار

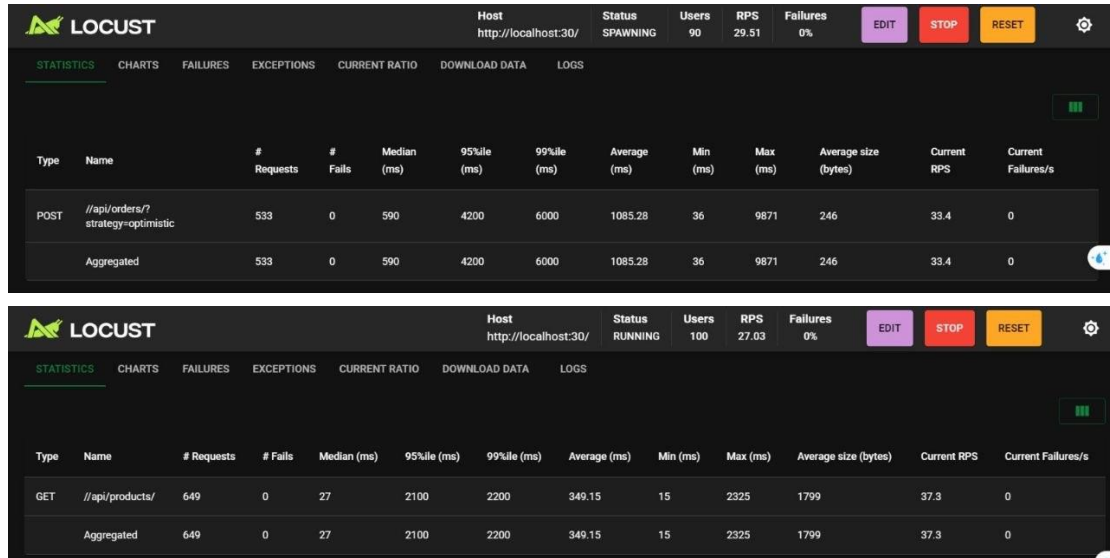
المعالجة غير المتزامنة للمهام الثقيلة (Asynchronous Task Offloading)

كجزء أساسي من توزيع الحمل الحسابي، تم عزل المهام التي تستهلك وقتاً وموارد عالية (مثل معالجة أرقام المبيعات الكبيرة، بناء ملفات التقارير وجرّد الـ CSV، وإرسال بريد التأكيد الإلكتروني) بعيداً عن دورة حياة طلب المستخدم (Request-Response Cycle).

يتم تمرير هذه المهام فوراً إلى **Celery Workers** لتعمل في الخلفية (Background Workers) عبر طوابير منظمة في Redis، مما يضمن بقاء خوادم الويب خفيفة، سريعة، ومتفرّغة تماماً لخدمة طلبات التصفح والشراء اللحظية للمستخدمين.

ثامنا :

تم تجربة الخادم على 100 مستخدم لعرض المنتجات وطلب اوردنر وقمنا بالتجربة على LOCUST



Products

Locust Test Report

During: 8/17/2026, 4:31:00 PM - 8/17/2026, 4:31:41 PM (41 seconds)

Target Host: http://127.0.0.1:30

Script: locustfile_stress_test.py

Request Statistics



Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /api/products/ [cached]	1178	0	83.83	23	386	800	28.5	0
GET	GET /api/products/id/ [DB direct]	751	0	82.11	21	353	81.58	18.17	0
GET	GET /api/products/most_viewed/ [cached]	766	0	79.79	20	321	288	18.51	0
GET	GET /api/products/trending/ [cached]	782	0	79.27	20	292	84	18.44	0
Aggregated		3456	0	81.49	20	388	388.27	83.82	0

Response Time Statistics

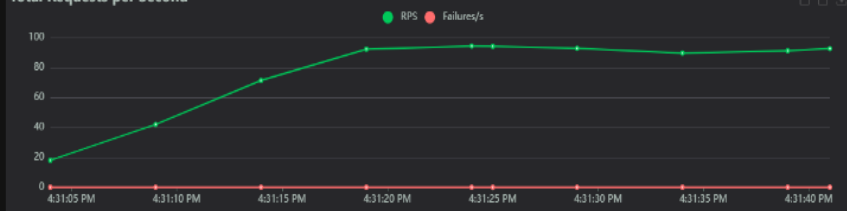
Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /api/products/ [cached]	81	77	97	120	180	220	280	370
GET	GET /api/products/id/ [DB direct]	81	75	92	120	180	230	280	350
GET	GET /api/products/most_viewed/ [cached]	68	73	92	120	170	200	270	320
GET	GET /api/products/trending/ [cached]	80	72	87	120	170	210	260	290
Aggregated		80	75	92	120	170	220	280	370

Failures Statistics

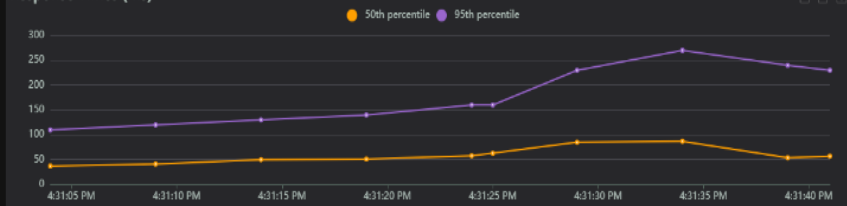
# Failures	Method	Name	Message
------------	--------	------	---------

Charts

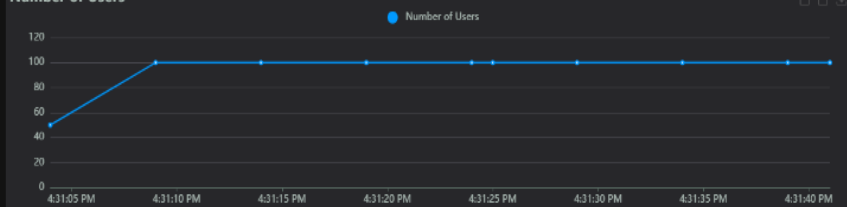
Total Requests per Second



Response Times (ms)



Number of Users



Final ratio

Ratio Per Class

- 100.0% ScenarioA_ProductReads
 - 22.2% getMostViewedProducts
 - 22.2% getProductById
 - 33.3% getProductListCached
 - 22.2% getTrendingProducts

Total Ratio

- 100.0% ScenarioA_ProductReads
 - 22.2% getMostViewedProducts
 - 22.2% getProductById
 - 33.3% getProductListCached
 - 22.2% getTrendingProducts

Create Order with Atomic

Locust Test Report

During: 6/17/2026, 5:36:34 PM - 6/17/2026, 5:36:46 PM (12 seconds)
Target Host: http://127.0.0.1:30
Script: locustfile_stress_test.py

Request Statistics



Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	POST /api/orders/ [atomic]	462	0	600.71	39	3383	153	38.19	0
Aggregated		462	0	600.71	39	3383	153	38.19	0

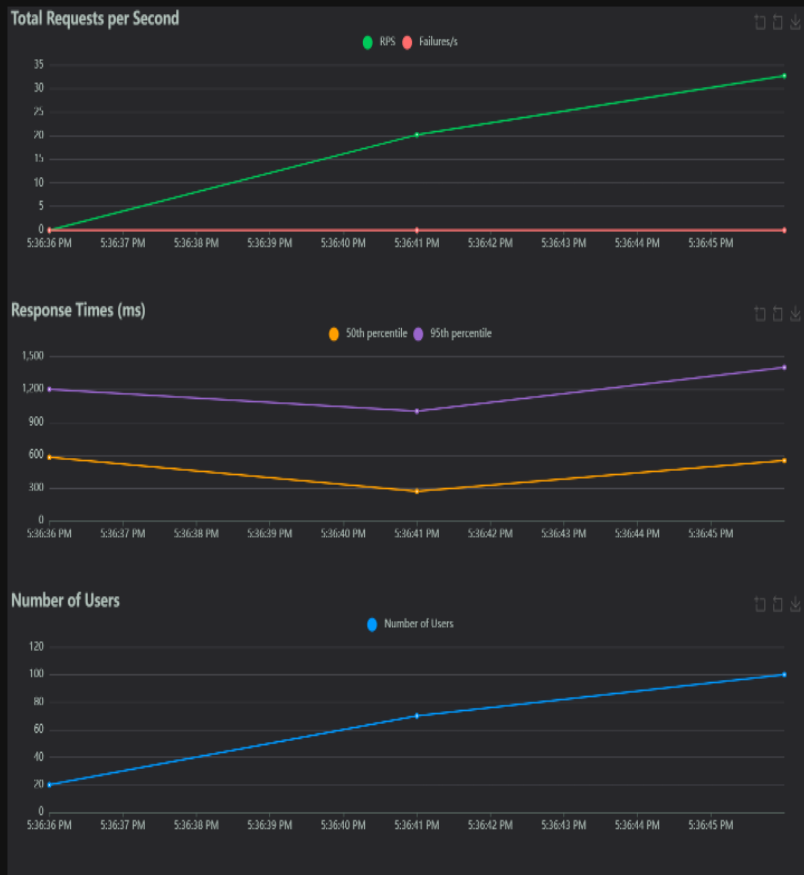
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	POST /api/orders/ [atomic]	540	620	710	840	1200	1400	2100	3400
Aggregated		540	620	710	840	1200	1400	2100	3400

Failures Statistics

# Failures	Method	Name	Message

Charts



Final ratio

Ratio Per Class

- 100.0% ScenarioB_OrderCreates_Atomic
 - 100.0% createOrderAtomic

Total Ratio

- 100.0% ScenarioB_OrderCreates_Atomic
 - 100.0% createOrderAtomic

Create Order with Pessimistic

Locust Test Report

During: 6/17/2026, 5:39:58 PM - 6/17/2026, 5:40:23 PM (25 seconds)

Target Host: http://127.0.0.1:30

Script: locustfile_stress_test.py

Request Statistics



Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	POST /api/orders/ [pessimistic]	882	0	1169.84	37	4730	153	36.38	0
Aggregated		882	0	1169.84	37	4730	153	36.38	0

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	POST /api/orders/ [pessimistic]	1200	1400	1600	1700	2100	2400	3100	4700
Aggregated		1200	1400	1600	1700	2100	2400	3100	4700

Failures Statistics

# Failures	Method	Name	Message

generate Daily Report

The screenshot shows the Postman interface for a Django API project. The left sidebar displays a collection of API endpoints, including 'Generate Daily Report' and 'Generate Weekly Report'. The main panel shows the details of a POST request to the endpoint `http://{{BASE_URL}}:{{PORT}}/api/reports/daily/`. The request is labeled 'Generate Daily Report'. The response is a JSON object with an error message: `{ "error": "The daily report is already being generated. Please check your background tasks." }`. The status bar indicates a 429 status code: '429 Too Many Requests'.

Global: E-Com GET GET GET GET GET GET POST POST POST POST POST Gene + No environment

E-Commerce Stress Test — Req 9 > Generate Reports > **Generate Daily Report** Save Share

POST `http://{{BASE_URL}}:{{PORT}}/api/reports/daily/` Send

Overview Params Authorization Headers 11 Body Scripts Settings Cookies

Generate Daily Report

[View complete documentation →](#)

`http://{{BASE_URL}}:{{PORT}}/api/reports/daily/`

Add request description...

Code Snippet

cURL `curl --location --request POST 'http://127.0.0.1:30/api/reports/daily/' \`

Body Cookies Headers 11 Test Results **429 Too Many Requests** 710 ms · 424 B Save Response

{ JSON Preview Debug with AI

```
1 {
2   "error": "The daily report is already being generated. Please check your background tasks."
3 }
```

Online Console Terminal 0 0 Import Complete

The screenshot shows the Postman interface for a Django API project. The left sidebar displays a collection of API endpoints, including 'Generate Daily Report' and 'Generate Weekly Report'. The main panel shows the details of a POST request to the endpoint `http://{{BASE_URL}}:{{PORT}}/api/reports/daily/`. The request is labeled 'Generate Daily Report'. The response is a JSON object with a success message: `{ "success": "Daily report generation has started in the background." }`. The status bar indicates a 202 status code: '202 Accepted'.

Global: E-Com GET GET GET GET GET GET POST POST POST POST POST Gene + No environment

E-Commerce Stress Test — Req 9 > Generate Reports > **Generate Daily Report** Save Share

POST `http://{{BASE_URL}}:{{PORT}}/api/reports/daily/` Send

Overview Params Authorization Headers 11 Body Scripts Settings Cookies

Generate Daily Report

[View complete documentation →](#)

`http://{{BASE_URL}}:{{PORT}}/api/reports/daily/`

Add request description...

Code Snippet

cURL `curl --location --request POST 'http://127.0.0.1:30/api/reports/daily/' \`

Body Cookies Headers 11 Test Results **202 Accepted** 293 ms · 391 B Save Response

{ JSON Preview Visualize

```
1 {
2   "success": "Daily report generation has started in the background."
3 }
```

Online Console Terminal 0 0 Import Complete

generate weekly report

The screenshot shows the Postman interface for a Django API project. The left sidebar displays a collection of API requests, including 'Generate Weekly Report'. The main panel shows the details of the selected request, which is a POST request to the endpoint `http://{{BASE_URL}}:{{PORT}}/api/reports/weekly/`. The request is successful, returning a 202 Accepted status with a response time of 342 ms and a body of `{ "success": "Weekly report generation started. It will be emailed shortly." }`.

Generate Weekly Report

`http://{{BASE_URL}}:{{PORT}}/api/reports/weekly/`

Add request description...

Code Snippet

```
curl --location --request POST 'http://127.0.0.1:38/api/reports/weekly/' \
```

Body: `{ "success": "Weekly report generation started. It will be emailed shortly." }`

The screenshot shows the Postman interface for the same Django API project. The left sidebar displays the collection of API requests. The main panel shows the details of the selected request, which is a POST request to the endpoint `http://{{BASE_URL}}:{{PORT}}/api/reports/weekly/`. The request is failed, returning a 429 Too Many Requests status with a response time of 84 ms and a body of `{ "error": "The weekly report is already being generated and emailed. Please wait." }`.

Generate Weekly Report

`http://{{BASE_URL}}:{{PORT}}/api/reports/weekly/`

Add request description...

Code Snippet

```
curl --location --request POST 'http://127.0.0.1:38/api/reports/weekly/' \
```

Body: `{ "error": "The weekly report is already being generated and emailed. Please wait." }`